

《移动终端软件开发技术》

Mobile Application Development

4. Android广播与通知

谢坤
计算机学院（国家示范性软件学院）

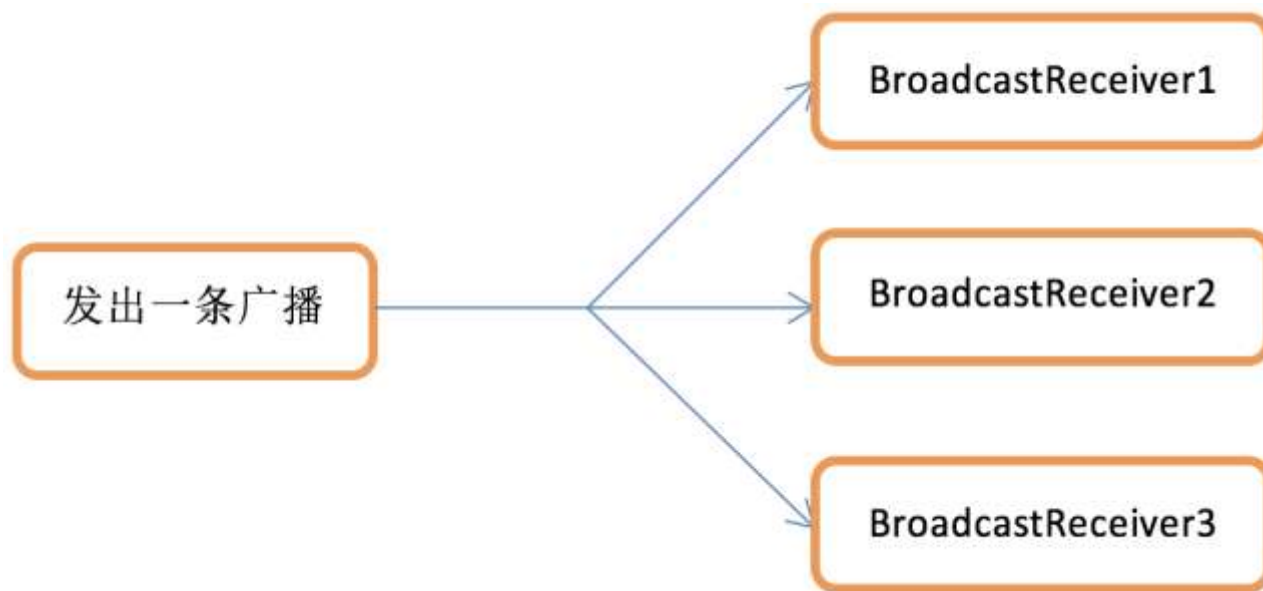
1 广播机制简介

1.1 广播机制

- 广播消息
 - 利用Intent发送广播消息
 - 应用程序和Android系统均可使用Intent发送广播消息
 - 消息内容可以是与应用程序密切相关的数据信息，也可以是Android的系统信息
 - 网络连接变化、电池电量变化、接收到短信、系统设置变化等
 - 利用BroadcastReceiver接收广播消息
 - 广播分类
 - 标准广播和有序广播

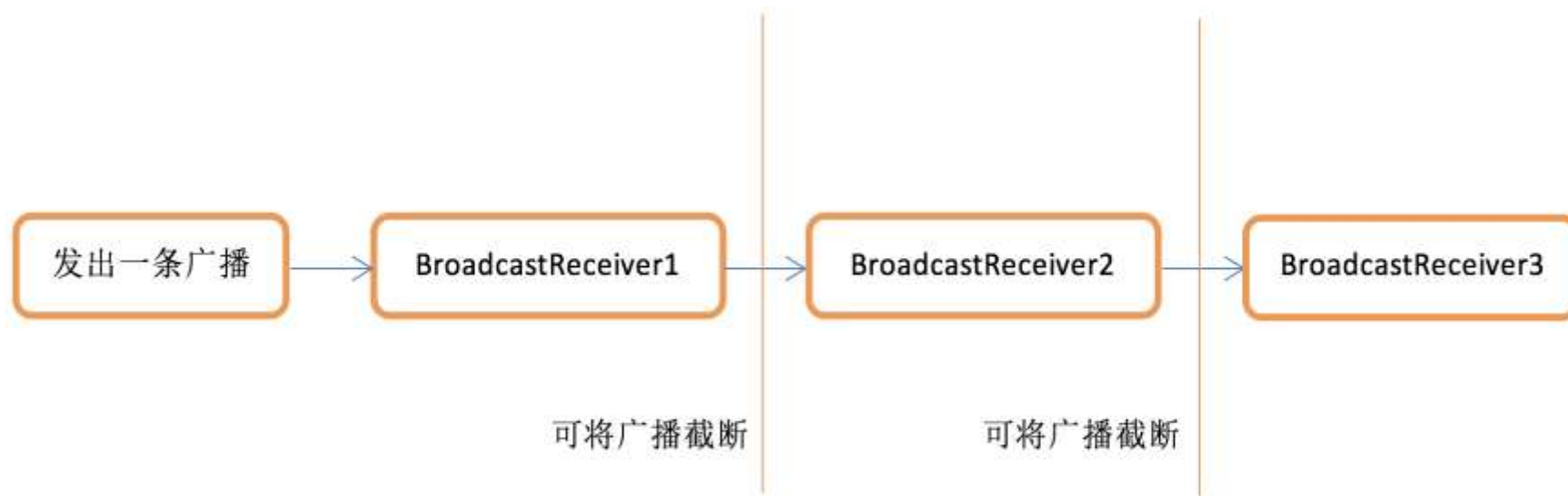
1.2 标准广播

- 标准广播 (normal broadcasts)
 - 异步执行，广播发出之后所有的BroadcastReceiver几乎会在同一时刻接收，无先后顺序。
 - 效率较高，但无法被截断。



1.3 有序广播

- 有序广播 (ordered broadcasts)
 - 同步执行，广播发出后同时刻只有一个BroadcastReceiver接收，其逻辑执行完毕后广播才会继续传递，有先后顺序。
 - 优先级高的BroadcastReceiver先收到广播消息，且前面的可以截断广播传递，此时后面的则无法接收。



2 接收系统广播

2.1 接收系统广播

- 系统广播
 - 系统级别的广播，用于传递系统的各类状态信息
 - 可根据需求自由注册BroadcastReceiver，当有相应的广播发出时，相应的BroadcastReceiver即可收到该广播，并可在内部进行逻辑处理
- 注册BroadcastReceiver的方式：
 - 在代码中注册：动态注册
 - 在AndroidManifest.xml中注册：为静态注册

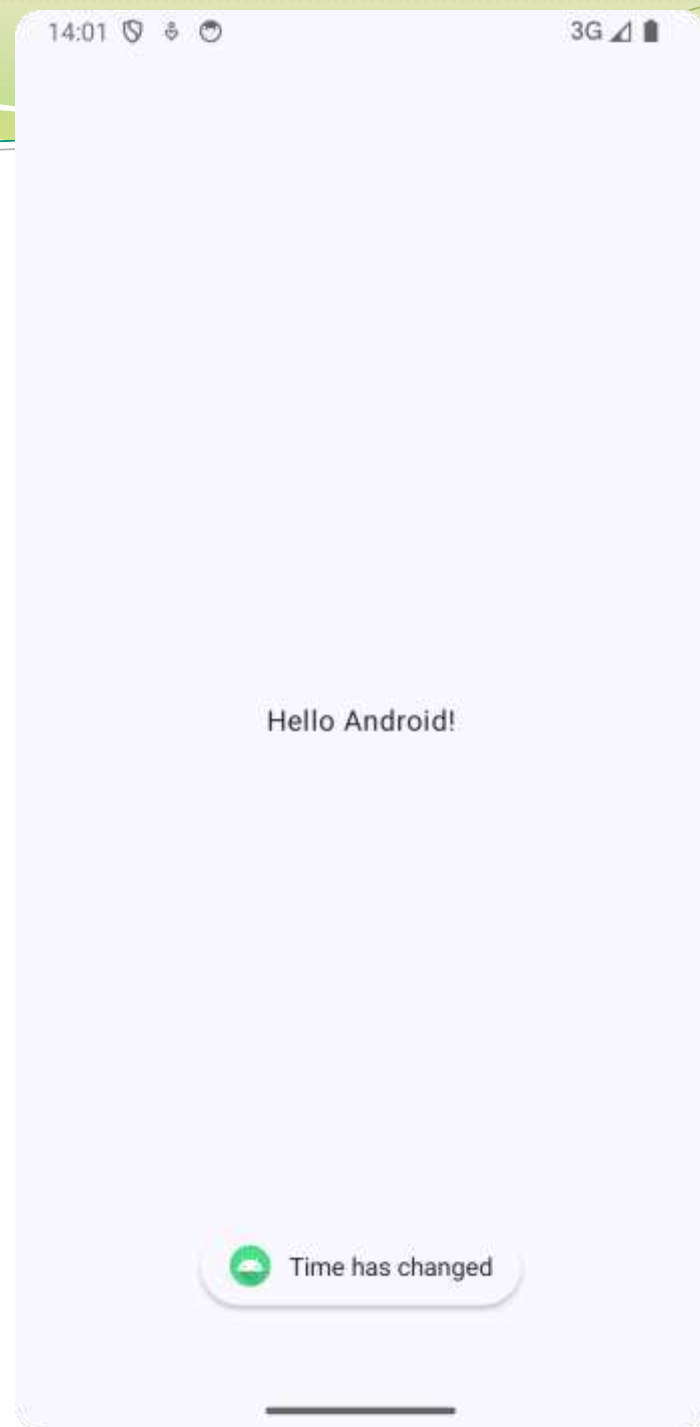
2.2 动态注册

```
class MainActivity : ComponentActivity() {  
    private lateinit var timeChangeReceiver: TimeChangeReceiver  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {.....}  
        val intentFilter = IntentFilter()  
        intentFilter.addAction("android.intent.action.TIME_TICK")  
        timeChangeReceiver = TimeChangeReceiver()  
        registerReceiver(timeChangeReceiver, intentFilter)  
    }  
  
    inner class TimeChangeReceiver : BroadcastReceiver() {  
        override fun onReceive(context: Context, intent: Intent) {  
            Toast.makeText(context, "Time has changed", Toast.LENGTH_SHORT).show()  
        }  
    }  
  
    override fun onDestroy() {  
        super.onDestroy()  
        unregisterReceiver(timeChangeReceiver)  
    }  
}
```

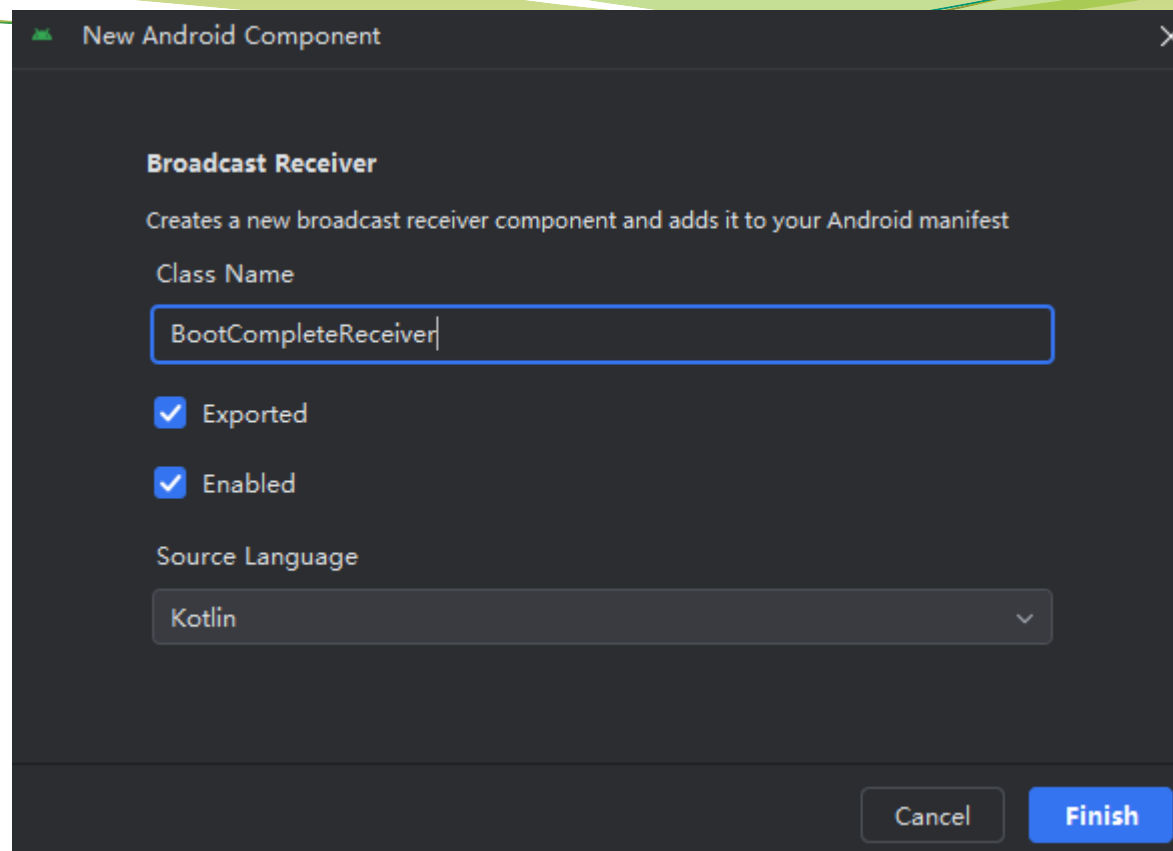
```
@Composable  
fun Greeting(name: String, modifier: Modifier = Modifier) {.....}
```

```
@Preview(showBackground = true)
```

```
@Composable  
fun GreetingPreview() {.....}
```

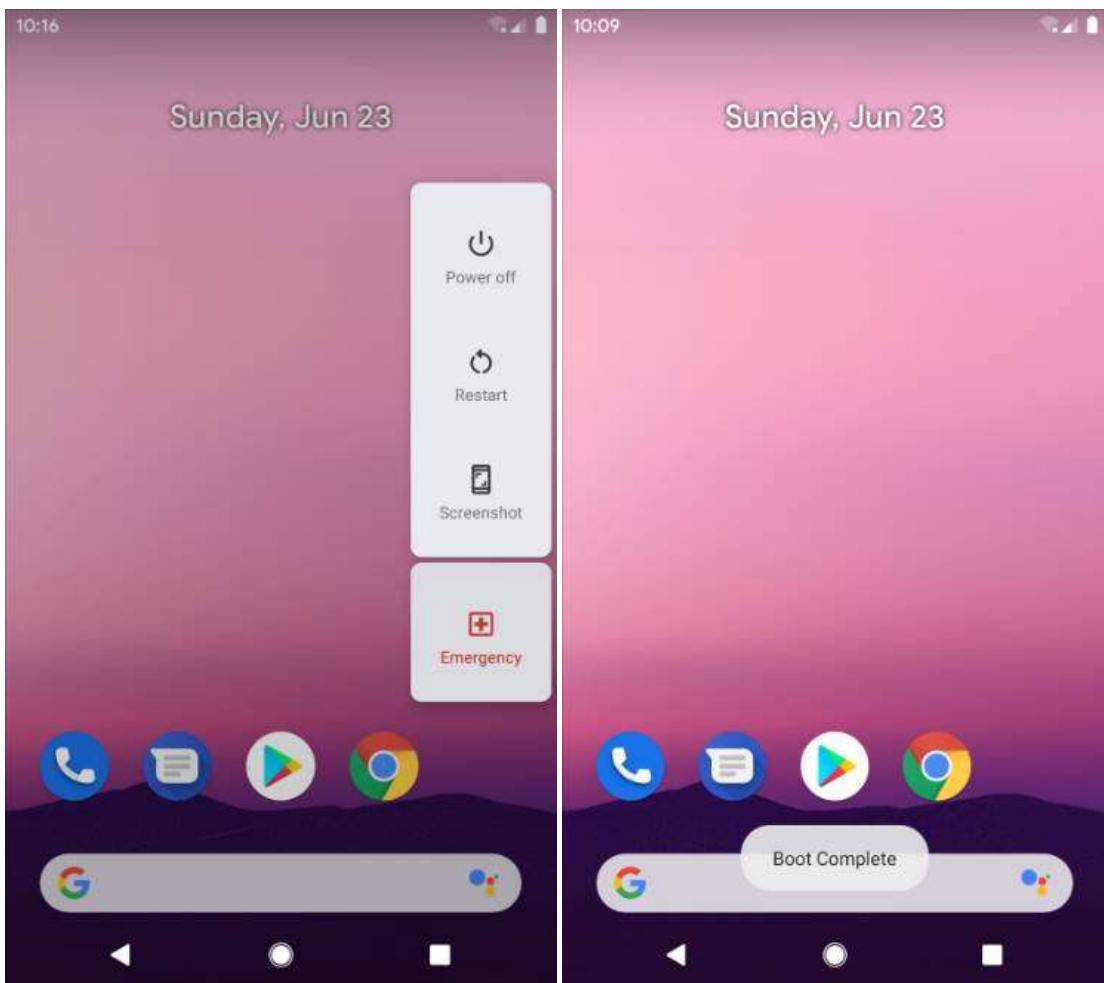


2.3 静态注册



```
class BootCompleteReceiver : BroadcastReceiver() {  
    override fun onReceive(context: Context, intent: Intent) {  
        Toast.makeText(context, "Boot Complete", Toast.LENGTH_LONG).show()  
    }  
}
```

2.3 静态注册



```
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
```

```
<application
```

```
    android:allowBackup="true"
```

```
    android:dataExtractionRules="@xml/data_extraction_rules"
```

```
    android:fullBackupContent="@xml/backup_rules"
```

```
    android:icon="@mipmap/ic_launcher"
```

```
    android:label="@string/app_name"
```

```
    android:roundIcon="@mipmap/ic_launcher_round"
```

```
    android:supportsRtl="true"
```

```
    android:theme="@style/Theme.CBroadcastTest"
```

```
    tools:targetApi="31">
```

```
    <receiver
```

```
        android:name=".BootCompleteReceiver"
```

```
        android:enabled="true"
```

```
        android:exported="true">
```

```
        <intent-filter>
```

```
            <action android:name="android.intent.action.BOOT_COMPLETED" />
```

```
        </intent-filter>
```

```
    </receiver>
```

```
    <activity
```

```
        android:name=".MainActivity"
```

3 发送自定义广播

3.1 发送标准广播

```
class MyBroadcastReceiver : BroadcastReceiver() {  
  
    override fun onReceive(context: Context, intent: Intent) {  
        Toast.makeText(  
            context,  
            "received in MyBroadcastReceiver",  
            Toast.LENGTH_SHORT).show()  
        abortBroadcast()  
    }  
}
```

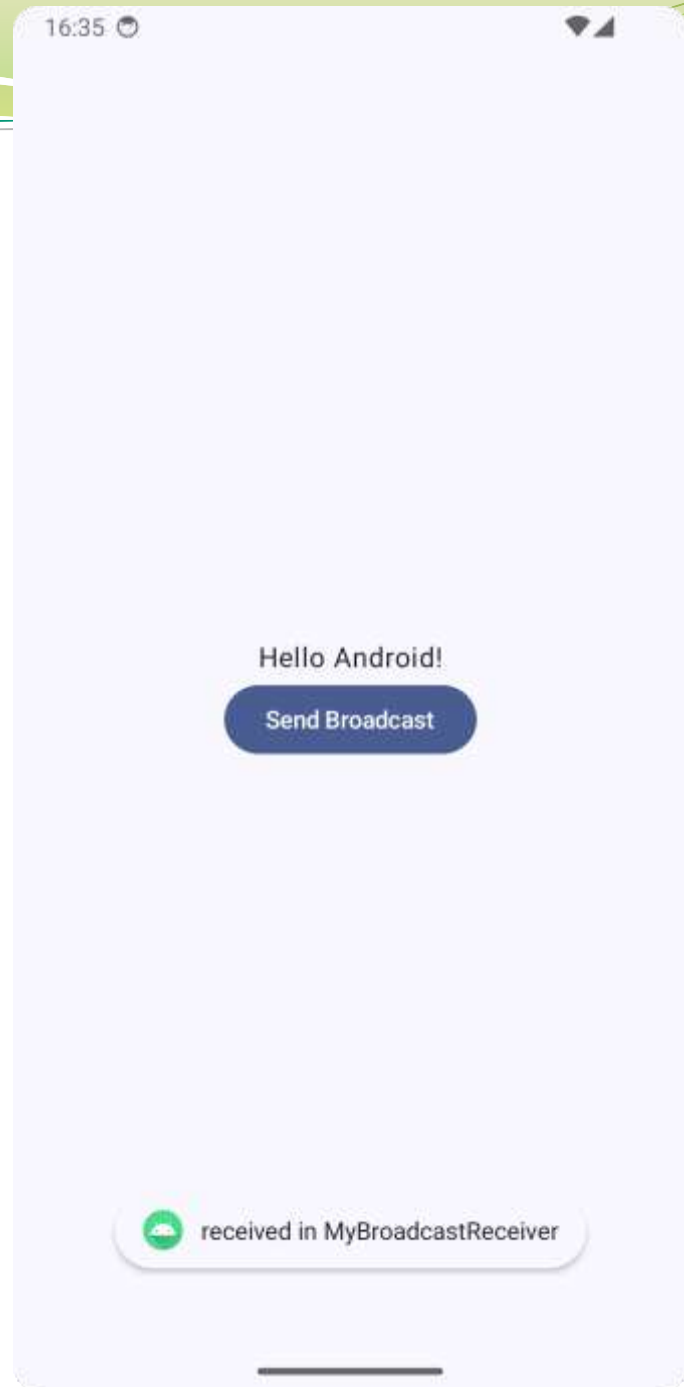
```
<receiver  
    android:name=".MyBroadcastReceiver"  
    android:enabled="true"  
    android:exported="true">  
    <intent-filter>  
        <action android:name="com.example.cbroadcasttest.MY_BROADCAST"/>  
    </intent-filter>  
</receiver>
```

3.1 发送标准广播

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current

    Column(
        modifier = modifier.fillMaxSize(),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Hello $name!",
            modifier = Modifier
        )

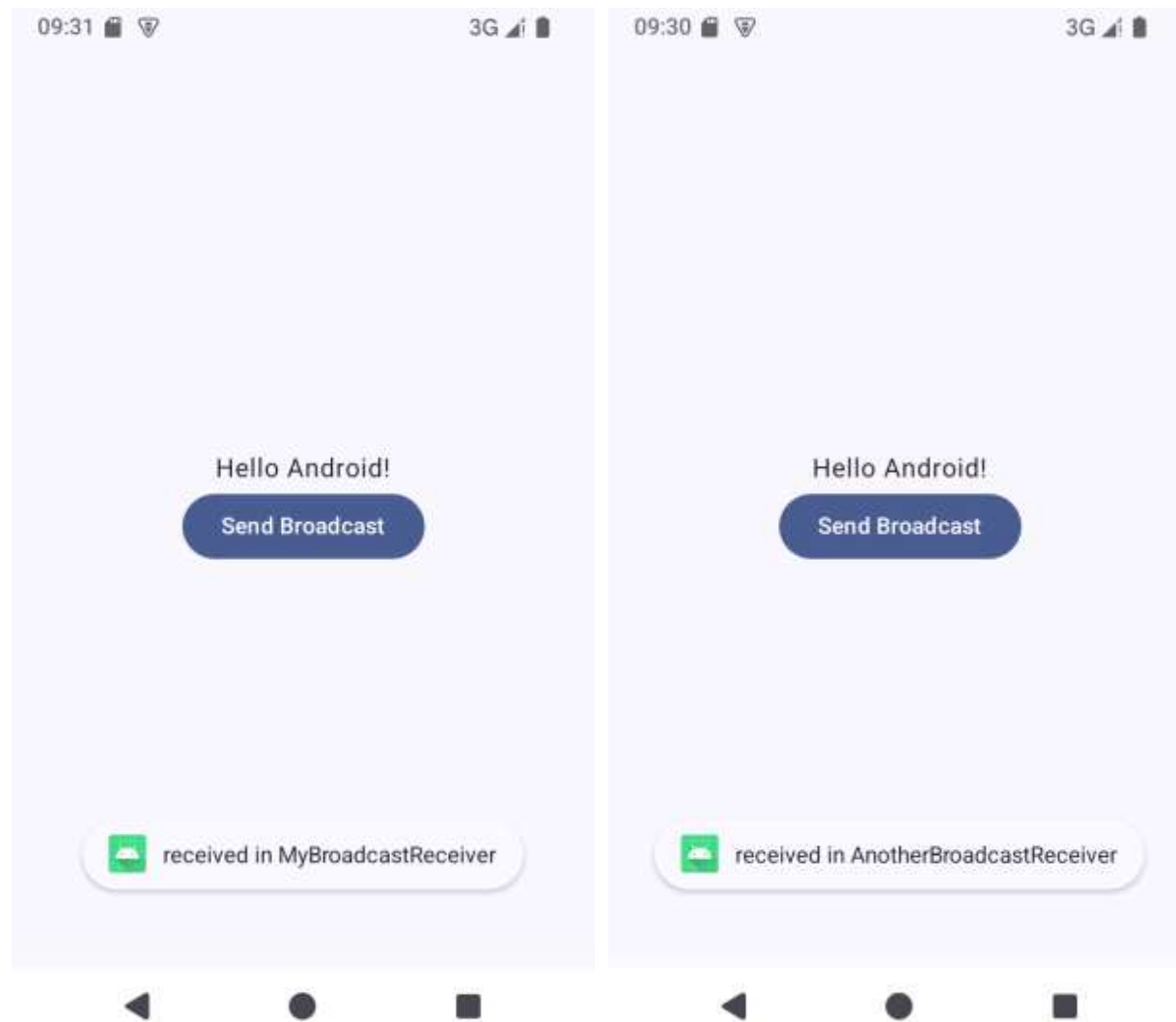
        Button(onClick = {
            val intent = Intent("com.example.cbroadcasttest.MY_BROADCAST")
            intent.setPackage(context.packageName)
            context.sendBroadcast(intent)
        }) {
            Text(stringResource(R.string.button_text))
        }
    }
}
```



3.2 发送有序广播

```
class AnotherBroadcastReceiver : BroadcastReceiver() {  
    override fun onReceive(context: Context, intent: Intent) {  
        Toast.makeText(  
            context,  
            "received in AnotherBroadcastReceiver",  
            Toast.LENGTH_SHORT  
        ).show()  
    }  
}
```

```
<receiver  
    android:name=".AnotherBroadcastReceiver"  
    android:enabled="true"  
    android:exported="true">  
    <intent-filter>  
        <action android:name="com.example.cbroadcasttest.MY_BROADCAST" />  
    </intent-filter>  
</receiver>
```

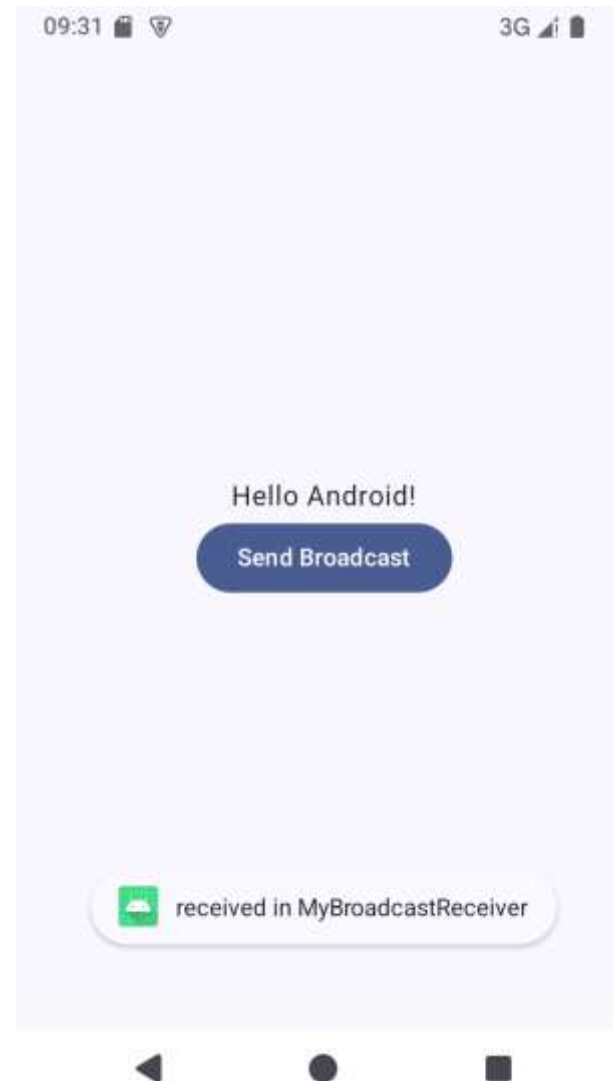


3.2 发送有序广播

```
Button(onClick = {  
    val intent = Intent("com.example.cbroadcasttest.MY_BROADCAST")  
    intent.setPackage(context.packageName)  
    context.sendOrderedBroadcast(intent, null)  
}) {  
    Text(stringResource(R.string.button_text))  
}
```

```
<receiver  
    android:name=".MyBroadcastReceiver"  
    android:enabled="true"  
    android:exported="true">  
    <intent-filter android:priority="100">  
        <action android:name="com.example.cbroadcasttest.MY_BROADCAST" />  
    </intent-filter>  
</receiver>
```

```
class MyBroadcastReceiver : BroadcastReceiver() {  
    override fun onReceive(context: Context, intent: Intent) {  
        Toast.makeText(  
            context,  
            "received in MyBroadcastReceiver",  
            Toast.LENGTH_SHORT  
        ).show()  
        abortBroadcast()  
    }  
}
```



4 Android通知

4.1 创建通知渠道

- 通知渠道：
 - 每条通知都属于一个对应的通知渠道。
 - 每个应用程序都可以自由地创建当前应用拥有哪些通知渠道。
 - 通知渠道的控制权掌握在用户手上。
 - 用户可以自由地选择这些通知渠道的重要程度。

4.1 创建通知渠道

- 通知渠道划分：
 - 需认真分析通知类型
 - 渠道创建后不可修改



4.1 创建通知渠道

```
1. val manager = getSystemService(Context.NOTIFICATION_SERVICE) as NotificationManager
```

```
1. if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
2.     val channel = NotificationChannel(channelId, channelName, importance)  
3.     manager.createNotificationChannel(channel)  
4. }
```

4.2 通知基本用法

- 创建通知：
 - 创建位置：Activity、BroadcastReceiver、Service
 - 创建步骤：
 1. 创建Notification对象
 2. 设置Notification对象
 3. 调用notify()显示通知

4.2 通知基本用法

```
1. val notification = NotificationCompat.Builder(context, channelId).build()
```

```
1. val notification = NotificationCompat.Builder(context, channelId)
2.   .setContentTitle("This is content title")
3.   .setContentText("This is content text")
4.   .setSmallIcon(R.drawable.small_icon)
5.   .setLargeIcon(BitmapFactory.decodeResource(getResources(), R.drawable.large_icon))
6.   .build()
```

```
1. manager.notify(1, notification)
```

4.2 通知基本用法

```
class MainActivity : ComponentActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)
```

```
        val manager = getSystemService(Context.NOTIFICATION_SERVICE) as  
            NotificationManager  
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
            val channel = NotificationChannel("normal", "Normal",  
                NotificationManager.IMPORTANCE_DEFAULT)  
            manager.createNotificationChannel(channel)  
        }
```

```
        .....  
        setContent {  
            .....  
        }  
    }
```

```
}
```

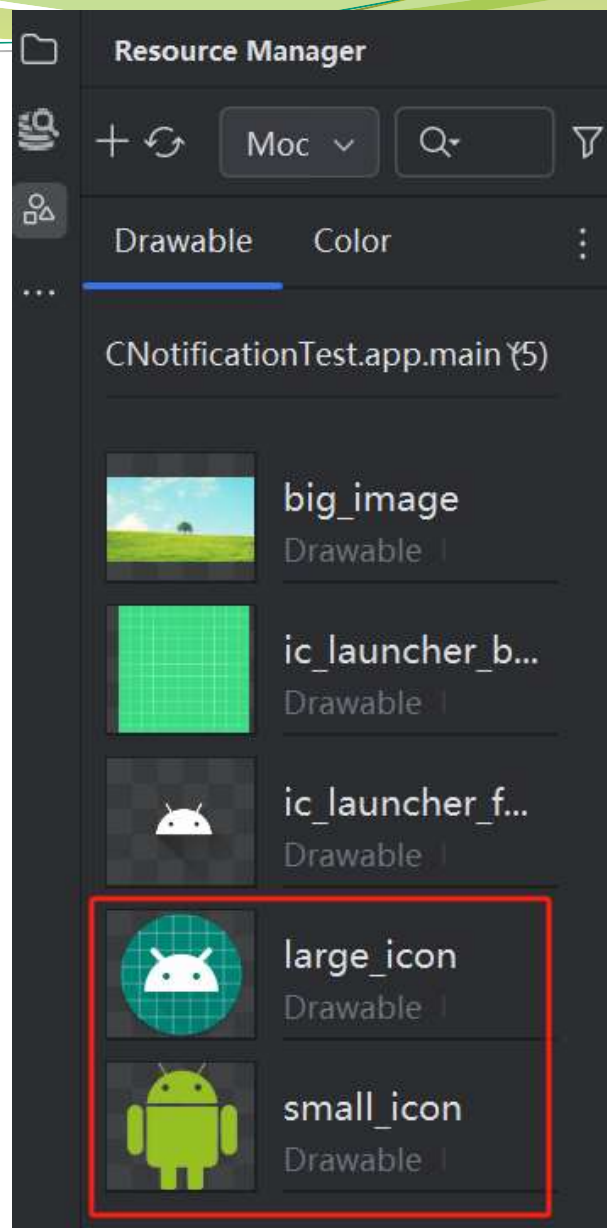
```
@Composable
```

```
fun Greeting(name: String, modifier: Modifier = Modifier) {  
    val context = LocalContext.current  
    Column(.....) {  
        Text(text = "Hello $name!")  
        Button(onClick = { sendBroadcast(context) })  
        { Text(stringResource(R.string.button_send)) }  
    }  
}
```

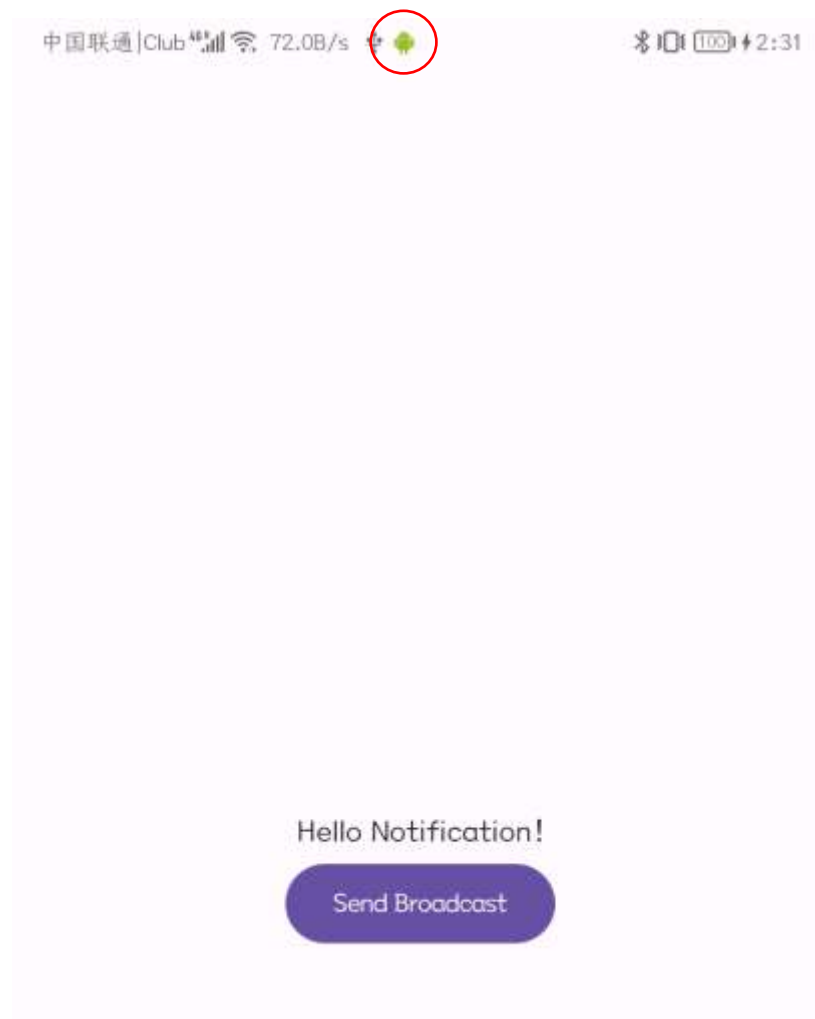
```
private fun sendBroadcast(context: Context) {  
    val intent = Intent("com.example.cnotificationtest.MY_BROADCAST")  
    intent.setPackage(context.packageName)  
    context.sendBroadcast(intent, null)  
}
```

4.2 通知基本用法

```
class MyBroadcastReceiver : BroadcastReceiver() {  
  
    override fun onReceive(context: Context, intent: Intent) {  
        val manager = context.getSystemService(  
            Context.NOTIFICATION_SERVICE  
        ) as NotificationManager  
        val notification = NotificationCompat.Builder(context, "normal")  
            .setContentTitle("This is content title")  
            .setContentText("This is short content text")  
            .setSmallIcon(R.drawable.small_icon)  
            .setLargeIcon(  
                BitmapFactory.decodeResource(context.resources,  
                    R.drawable.large_icon))  
            .build()  
        manager.notify(1, notification)  
    }  
}
```



4.2 通知基本用法



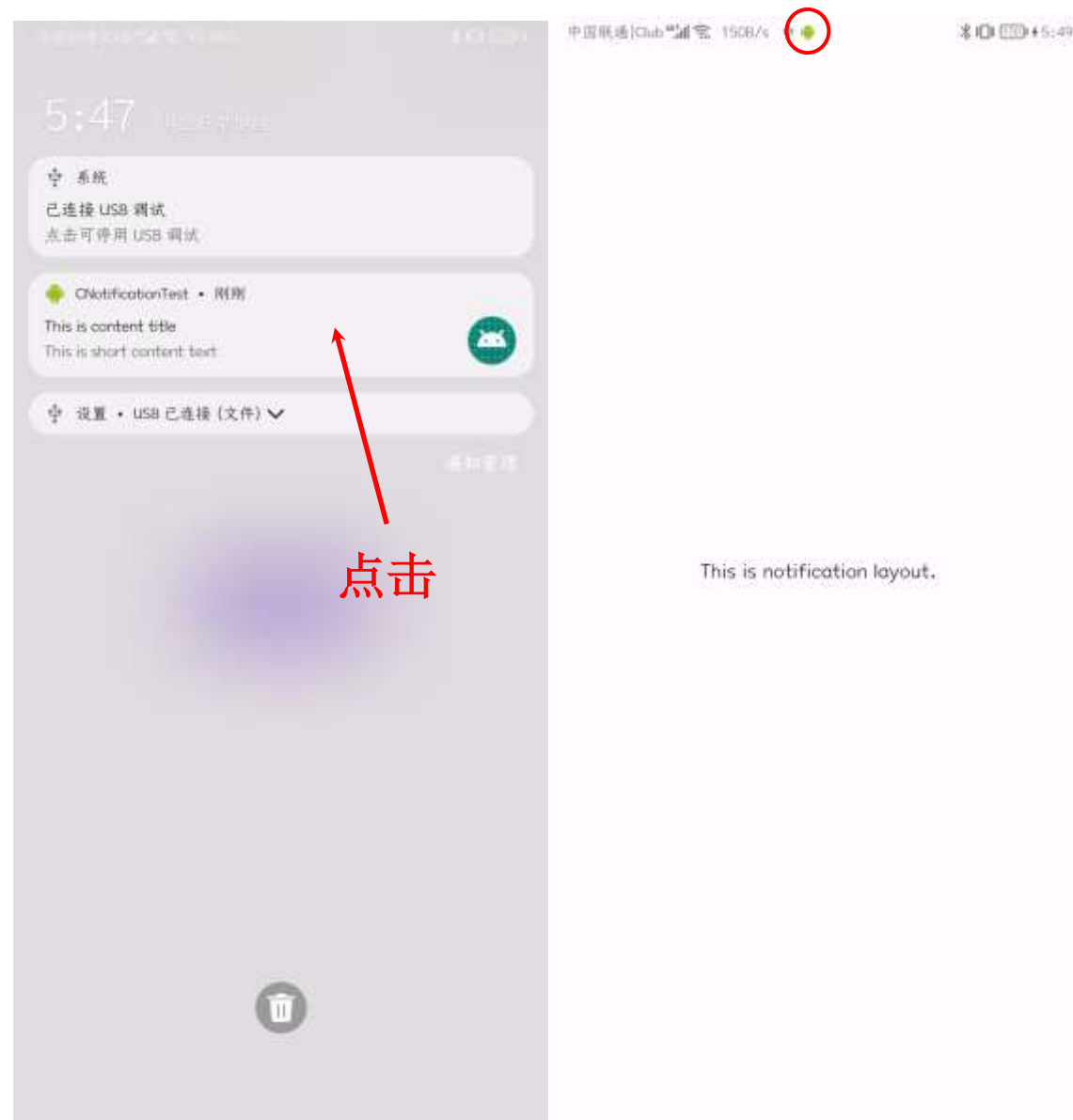
4.2 通知基本用法

- 通知点击效果
 - PendingIntent:
 - 延迟执行的Intent，在某个合适的时机执行某个动作。
 - 获取PendingIntent实例的静态方法：
 - getActivity()、getBroadcast()、getService()。
 - 4个参数。
 - setContentIntent():
 - 为通知设定PendingIntent。

4.2 通知基本用法

```
@Composable
fun Greeting2(modifier: Modifier = Modifier) {
    Column(.....) {
        Text(
            text = "This is notification layout.",
            textAlign = TextAlign.Center
        )
    }
}
```

```
class MyBroadcastReceiver : BroadcastReceiver() {
    override fun onReceive(
        context: Context, intent: Intent) {
        val anotherIntent = Intent(
            context,
            NotificationActivity::class.java
        )
        val pi = PendingIntent.getActivity(
            context, 0, anotherIntent,
            PendingIntent.FLAG_IMMUTABLE
        )
        .....
    }
}
```



4.2 通知基本用法

```
class MyBroadcastReceiver : BroadcastReceiver() {  
    override fun onReceive(context: Context, intent: Intent) {  
        .....  
        val notification = NotificationCompat.Builder(  
            context, "normal")  
        .....  
        .setAutoCancel(true)  
        .build()  
        manager.notify(1, notification)  
    }  
}
```

```
class NotificationActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        .....  
        val manager = getSystemService(  
            Context.NOTIFICATION_SERVICE)  
        as NotificationManager  
        manager.cancel(1)  
    }  
}
```

中国联通 | Club 4G 169B/s

6:01

This is notification layout.

4.3 通知高级用法

- 通知显示效果
 - `setStyle()` 方法
 - `NotificationCompat.Style` 参数
 - 富文本：包含长文本、图片等。

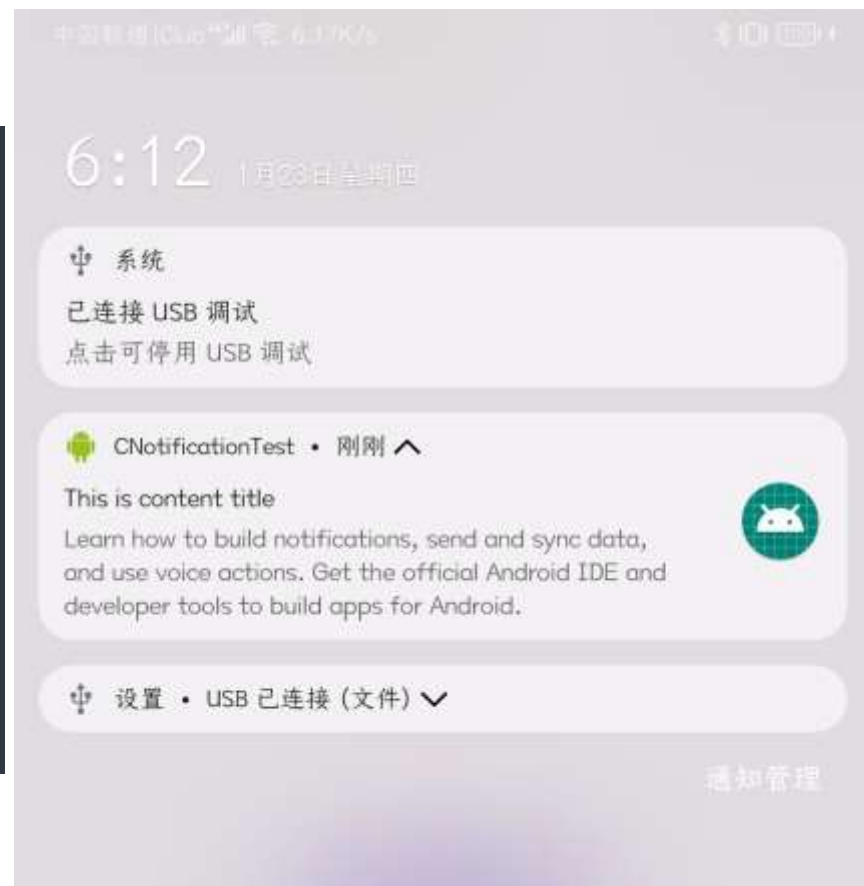
4.3 通知高级用法

```
class MyBroadcastReceiver : BroadcastReceiver() {  
    override fun onReceive(context: Context, intent: Intent) {  
        .....  
        val notification = NotificationCompat.Builder(context, "normal")  
            .setContentTitle("This is content title")  
            .setContentText(("Learn how to build notifications, " +  
                "send and sync data, and use voice actions. " +  
                "Get the official Android IDE and developer " +  
                "tools to build apps for Android."))  
        .....  
        .build()  
        manager.notify(1, notification)  
    }  
}
```



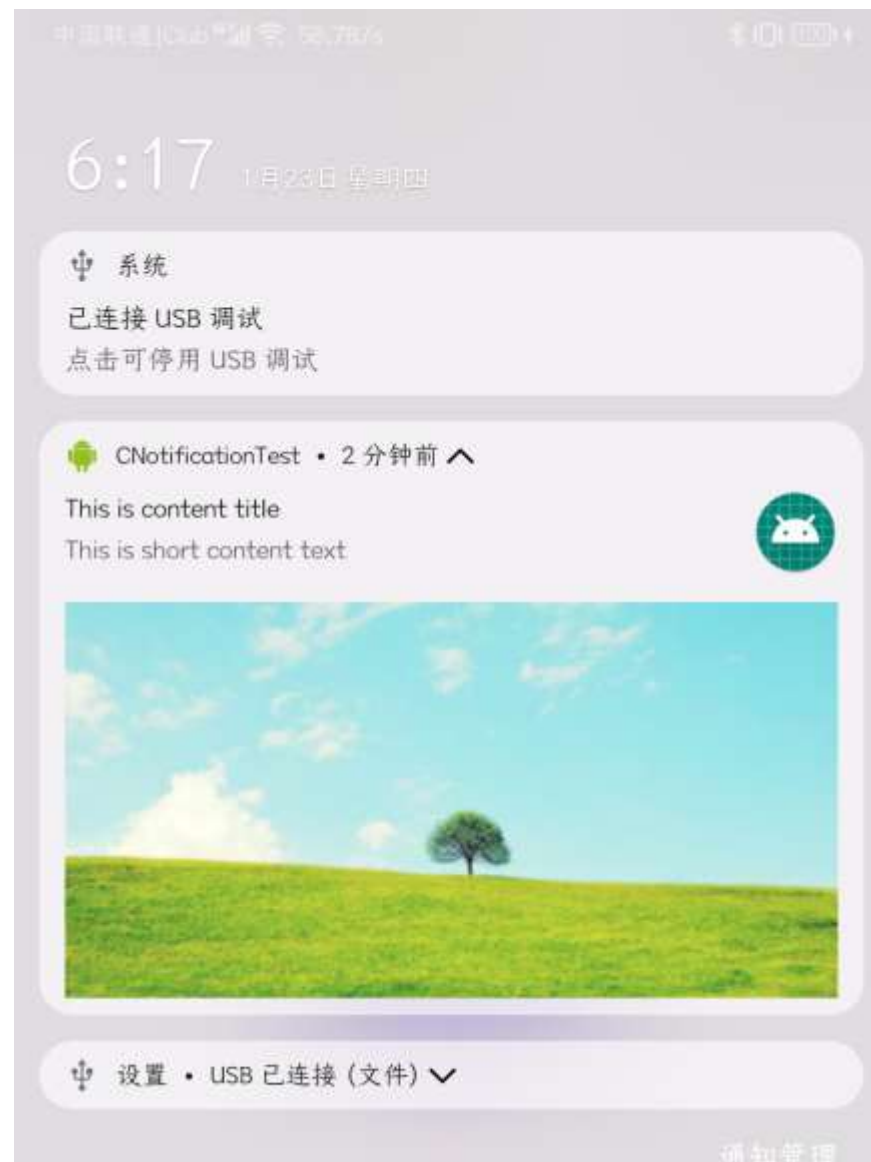
4.3 通知高级用法

```
override fun onReceive(context: Context, intent: Intent) {  
    .....  
    val notification = NotificationCompat.Builder(context, "normal")  
    .....  
    .setContentText("This is short content text")  
    .setStyle(NotificationCompat.BigTextStyle().bigText(  
        "Learn how to build notifications, send and sync data, " +  
        "and use voice actions. Get the official Android IDE " +  
        "and developer tools to build apps for Android."))  
    .....  
    .build()  
    manager.notify(1, notification)  
}
```



4.3 通知高级用法

```
override fun onReceive(context: Context, intent: Intent) {  
    .....  
    val notification = NotificationCompat.Builder(context, "normal")  
        .....  
        .setContentText("This is short content text")  
        .setStyle(NotificationCompat.BigPictureStyle()  
            .bigPicture(  
                BitmapFactory.decodeResource(  
                    context.resources,  
                    R.drawable.big_image  
                )  
            )  
        )  
        .....  
        .build()  
    manager.notify(1, notification)  
}
```



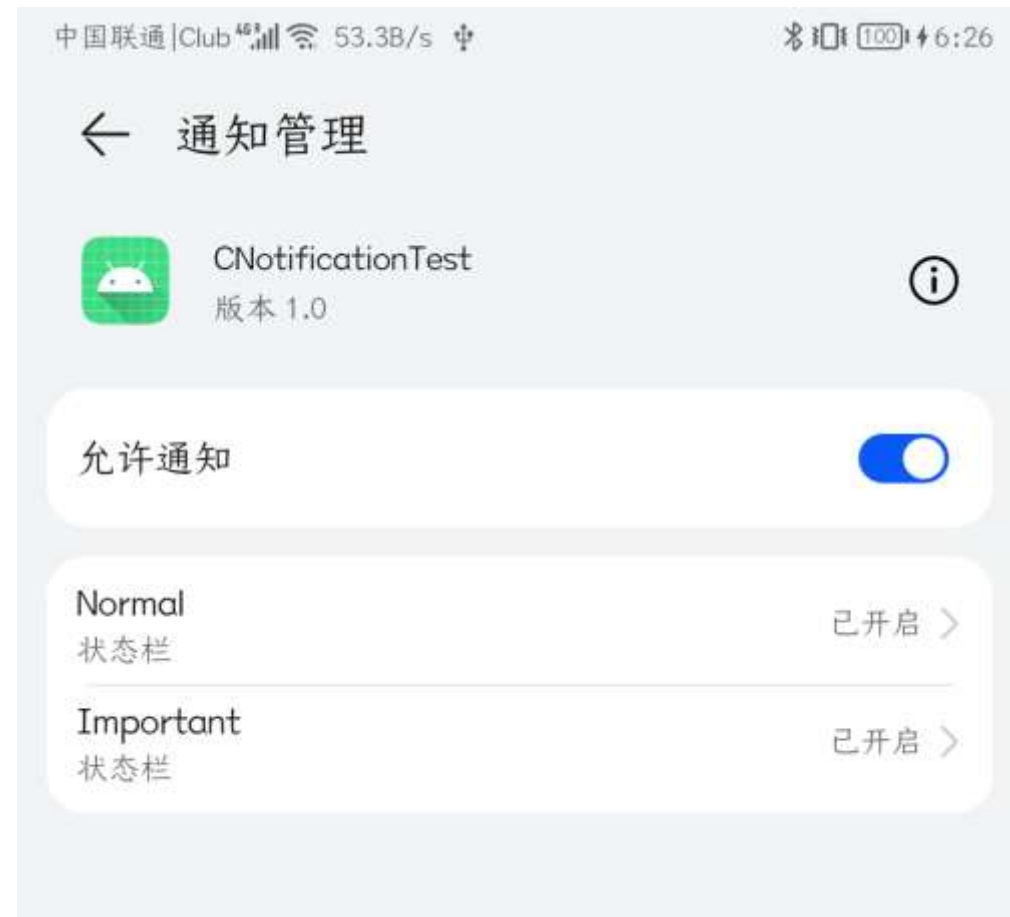
4.3 通知高级用法

- 通知渠道的重要等级设定
 - 对通知的行为的影响
 - 通知渠道的重要等级越高，发出的通知就越容易获得用户的注意。
 - 开发者只能在创建通知渠道的时候为它指定初始的重要等级，用户可以随时进行修改。

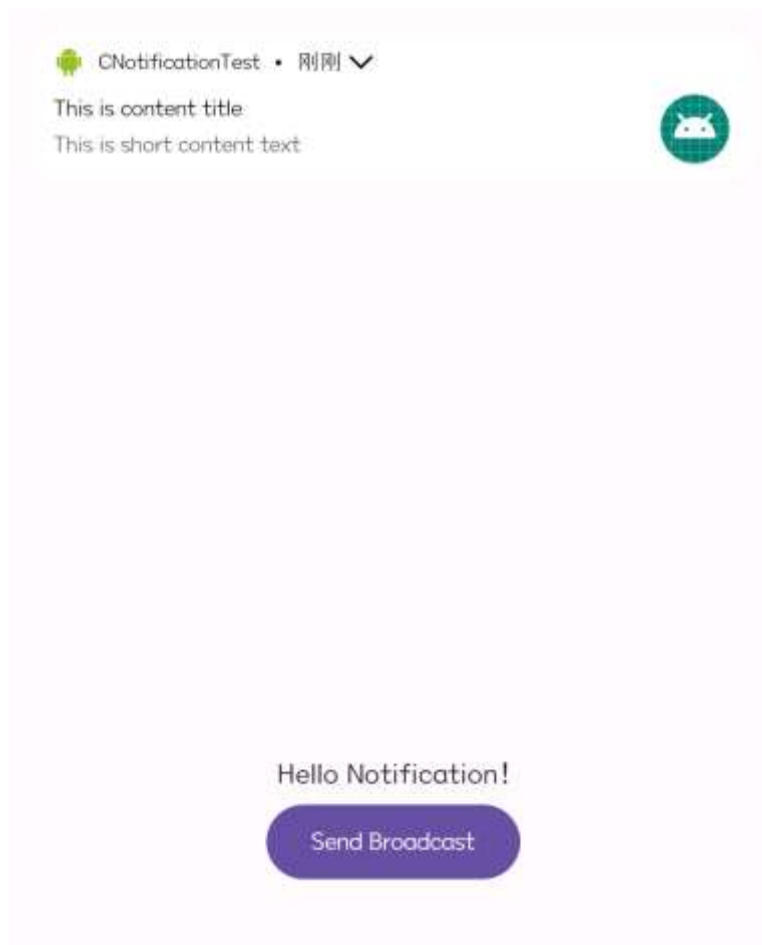
4.3 通知高级用法

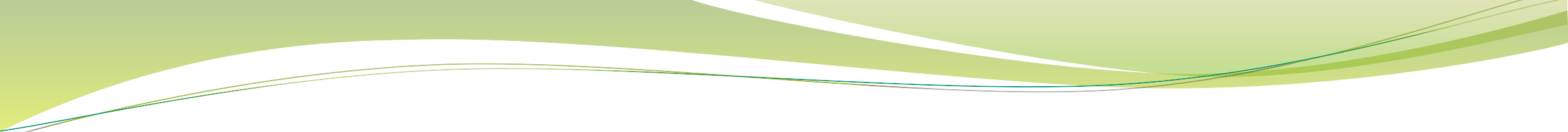
```
override fun onCreate(savedInstanceState: Bundle?) {
    .....
    val manager = getSystemService(Context.NOTIFICATION_SERVICE) as
        NotificationManager
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val channel1 = NotificationChannel("normal", "Normal",
            NotificationManager.IMPORTANCE_DEFAULT)
        manager.createNotificationChannel(channel1)
        val channel2 = NotificationChannel("important", "Important",
            NotificationManager.IMPORTANCE_HIGH)
        manager.createNotificationChannel(channel2)
    }
    .....
}
```

```
override fun onReceive(context: Context, intent: Intent) {
    .....
    val manager = context.getSystemService(Context.NOTIFICATION_SERVICE)
        as NotificationManager
    val notification = NotificationCompat.Builder(context, "important")
        .....
        .build()
    manager.notify(1, notification)
}
```



4.3 通知高级用法





感谢观看！

